Part A - Research Paper Study

Paper: PathRL: An End-to-End Path Generation Method for Collision Avoidance via Deep Reinforcement Learning (Yu et al., USTC, 2023)

I. Objective

The paper's main aim is to train a robot to navigate and avoid collisions by having the DRL policy output a whole path instead of raw velocity commands. This fixes the usual problem with DRL navigation, jerky speeds and shaky trajectories ,because the path is then followed smoothly by a separate path-tracking controller. The end goal is stable, smooth, human-friendly robot movement without sacrificing how often the robot actually reaches its target.

II. Domain

Autonomous mobile robot navigation and collision avoidance, using deep reinforcement learning. The work specifically targets navigation in crowded, dynamic settings with static obstacles, moving pedestrians, and other robots.

III. Scope of the Work

They build the full pipeline: a DRL policy (using the SAC algorithm) that generates a Bézier-curve path from three control points, plus curriculum learning to make training faster and a custom reward for smoothness. They test it against other DRL methods (PPO, SAC, PSD) and the classical TEB planner across static, pedestrian, and multi-agent scenarios in simulation. Finally they deploy it on a real Ackermann-steering robot to confirm it works outside simulation.

IV. Key Areas Open for Future Enhancement / Optimization

In dynamic scenes PathRL's paths are a bit longer and slower (higher LEN and TIME) because it prioritizes smoothness- trimming that trade-off is one open area. It also lacks the pedestrian-specific map input that PSD uses, so adding richer human-motion awareness could push success rates higher in crowds. Data association and mapping still lean on standard SLAM, so tighter integration of learning into perception is another direction.

V. Key Design Aspects

A. Deployment Platform

Trained in a custom 2D simulator and validated in a 3D Gazebo environment, then deployed on a real Ackermann-steering robot (Agilex Hunter 2.0 chassis, roughly 0.95 × 0.75 × 1.45 m). Compute is handled onboard by an RTX 3090. This mix of simulation and real hardware shows the policy transfers from sim to the real world.

B. Sensors & Types

The robot uses a 3D laser sensor with a 180° field of view to build its view of the surroundings.

The real-world platform specifically carries a 32-line 3D LiDAR. These feed an egocentric costmap of nearby obstacles and the robot's own shape.

C. Perception: Sensor Measurement Model & Motion Model

Perception is packaged as an 84 × 84 egocentric costmap plus a relative target pose (position and orientation to the goal). The motion side is modeled as a semi-MDP, where the robot commits to following a full path before the next decision, rather than reacting step by step. Three consecutive path-based frames are stacked as input so the policy "sees" motion over a longer time window.

D. Localization & Mapping

For the real-world deployment they rely on standard SLAM for mapping and localization (Pfilter, a LiDAR feature-filtering SLAM). Localization isn't the paper's focus — it's treated as a supporting module so the navigation policy can concentrate on path generation. The costmap that the policy uses is built on top of this localization layer.

E. Trajectory Planning

This is the core contribution: the DRL policy outputs three control points, which are fit into a smooth Bézier curve that becomes the path. To keep training manageable, they fix the longitudinal spacing of control points and only let the policy pick the horizontal values ("semi-discrete" action space). This keeps the path kinematically feasible and much smoother than raw velocity outputs.

F. Control Technique

Planning and control are deliberately decoupled - the DRL policy plans the path, and a separate path-following controller (with a fixed linear velocity) actually drives the robot. This split is what makes the motion predictable and lets controller parameters be tuned to each specific robot. The reward function adds curvature and straightness penalties so generated paths stay smooth and align with the robot's heading.

G. Communication Infrastructure

The paper doesn't emphasize IoT/cloud/Wi-Fi communication - everything runs onboard the robot with local compute (the RTX 3090). In the multi-agent tests, robots operate in a non-communicating way, avoiding each other purely through their own sensing rather than exchanging messages. So the design is intentionally self-contained rather than network-dependent.

VI. Components & Phases of Robotics — Design, Modelling & Implementation, and Comparative Analysis

Design & Modelling. PathRL is built around the classic robotics pipeline — perception, localization/mapping, planning, and control — but redraws the boundary between planning and control. Instead of the DRL policy spitting out low-level wheel velocities, it models navigation as a semi-MDP and outputs a *path* (a Bézier curve from three control points), which a separate tracking controller then follows. The reward is modelled from four parts - reaching the goal, avoiding collisions, low curvature, and straight alignment -so smoothness is baked into the design rather than patched on afterward.

Implementation. The policy is trained with Soft Actor-Critic (SAC), fed three stacked path-based frames (an 84×84 egocentric costmap plus relative target pose), and sped up with two-stage curriculum learning (simple static maps first, then complex dynamic crowds). It's implemented in a custom 2D simulator, validated in 3D Gazebo, and finally deployed on a real Ackermann robot using LiDAR-based SLAM for localization -showing the full sim-to-real path.

Comparative Analysis to Traditional Techniques. Compared with classical planners like TEB, ORCA, and the Social Force Model -which need heavy manual parameter tuning per scenario — PathRL learns its behaviour end-to-end and adapts across scenarios without that hand-tuning. Against other DRL methods that output raw velocities (PPO, SAC, PSD), it produces dramatically smoother motion: average steering-angle change drops from roughly 0.5–1.2 rad down to about 0.002–0.01 rad, and curvature is consistently the lowest, all without hurting success rate. The trade-off is honest - in dynamic scenes its paths are slightly longer and slower because it prioritises stable, smooth avoidance, which is generally what real deployments prefer.

Link to our implementation. Our notebook mirrors these same phases on the warehouse-bot domain: MCL and EKF-SLAM cover the probabilistic perception/localization the paper leans on, D* Lite handles the planning layer, and -echoing PathRL's core idea of swapping hand-derived models for learned ones -our Parts D and E replace classical components with reinforcement learning (active localization) and traditional ML (a Random-Forest localizer). This lets us directly compare model-based versus learning-based approaches, which is exactly the comparative lens the paper takes.

Part- B

Localization, SLAM, Planning and Learning for an Autonomous Warehouse Robot — simulation results and ROS 2 deployment

GROUP 11 — MEMBER DETAILS

S.No	NAME	BITS ID
1	AJAY KADAMBALA	2024AC05936
2	CHINTALURU DEEPIKA	2024AC05485
3	DEEPAK S P	2024AC05701
4	PRAVEENKUMAR T R	2024AC05334
5	SHUBHANJOY BISWAS	2024AD05076

SCOPE

This annex presents the simulation results for the warehouse-robot domain — a differential-drive AMR navigating narrow shelf aisles with wheel-encoder odometry, IMU, 2D LiDAR and an RGB-D camera. Five deliverables are implemented against the same domain and packaged as ROS 2 nodes exchanging standard interfaces over named topics, with QoS profiles and the REP-105 map → odom → base_link frame tree. The algorithms themselves are implemented in NumPy, which keeps the internals - particles, covariances, Q-tables -directly inspectable

A — Monte Carlo Localization (Particle Filter)

The robot starts *globally uncertain*: 500 particles are scattered uniformly over the entire 10 m × 12 m warehouse floor with random headings. Each timestep the particles are propagated through the Assignment-1 probabilistic odometry motion model, weighted by the likelihood of the range-bearing observations of the shelf-corner / QR landmarks (black squares), and resampled with low-variance systematic resampling whenever the effective sample size drops below N/2.

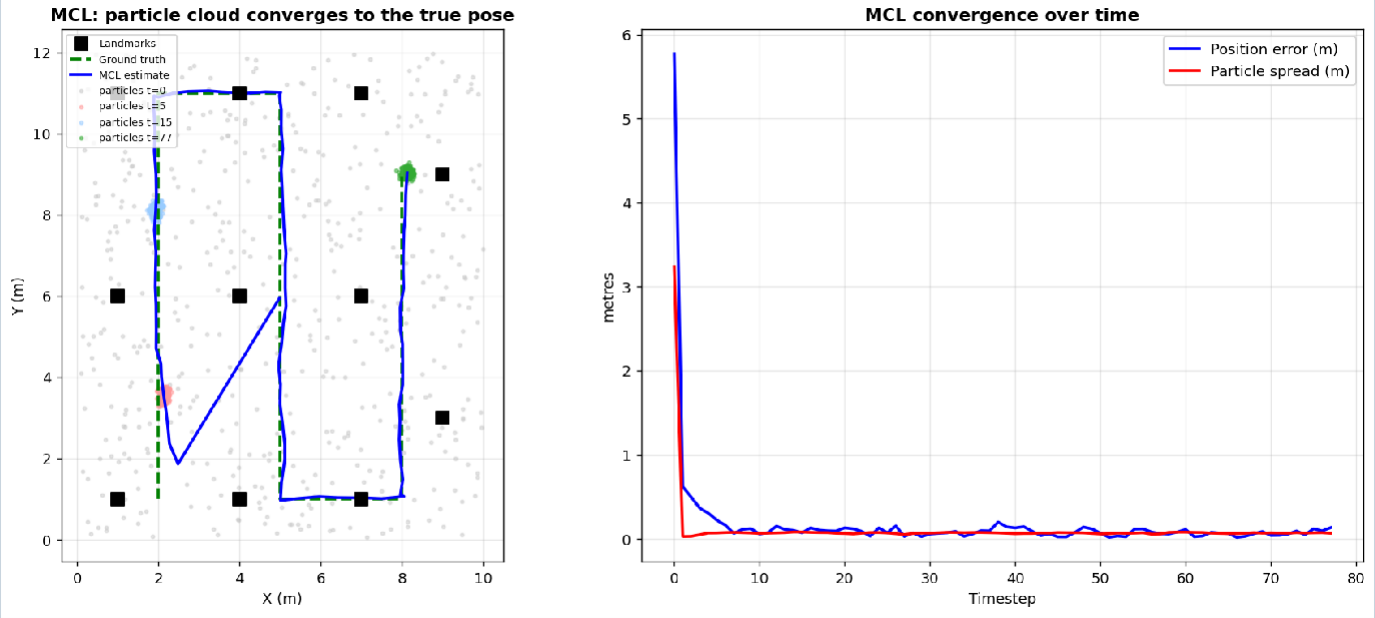


Figure 1 — `A_mcl.png`*, generated by* `Warehouse_bot_Assignment2_Group11.ipynb`*.*

DESCRIPTION & OBSERVATIONS

- **Left panel** — the particle cloud at t = 0, 5, 15 and the final step, collapsing onto the true pose; the green dashed line is ground truth and the blue line the MCL estimate.
- **Right panel** — position error and particle spread against time.
- **Result:** position error falls from **5.77 m to 0.14 m**; particle spread contracts from **3.24 m to 0.069 m**, i.e. the belief converges within a few sensor updates.
- **ROS 2 node:** `/mcl_localizer` — consumes `/odom` and `/landmark_detections`, publishes `/amcl_pose`, `/particlecloud` and TF `map → odom`.

B — Feature-based SLAM (EKF-SLAM)

Here the map is *unknown*: the robot must estimate its own pose and the landmark positions simultaneously in one augmented Gaussian state. The prediction step inflates the pose covariance through the motion Jacobian `G`; every range-bearing observation is fused via the measurement Jacobian `H`, and a newly-seen landmark is initialised into the state with high covariance that subsequent sightings tighten.

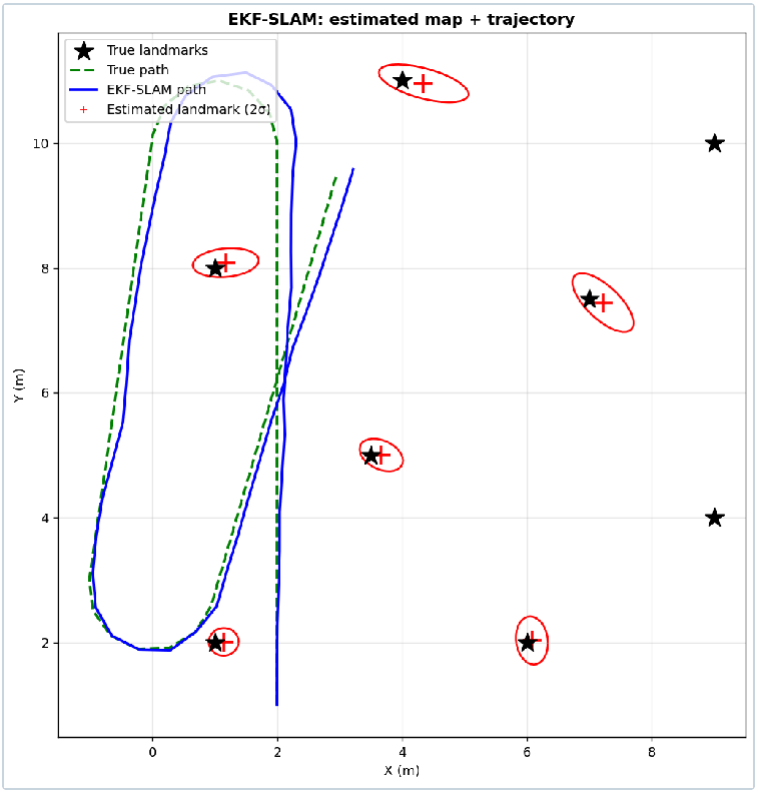


Figure 2 — `B_ekfslam.png`*, generated by* `Warehouse_bot_Assignment2_Group11.ipynb`.

DESCRIPTION & OBSERVATIONS

- Black stars are the true landmark positions, red crosses the EKF estimates, and the red ellipses their 2σ covariance — visibly shrinking as landmarks are re-observed.
- **Result: 6 of 8** landmarks mapped to a mean error of **0.196 m**, with a final pose error of **0.274 m**. The two unmapped landmarks never entered the 6 m sensor range on this single pass.
- The off-diagonal robot–landmark correlations in `Σ` are precisely what allow a later re-observation to correct earlier estimates — the mechanism behind loop closure.
- **ROS 2 node:** `/ekf_slam` — `/cmd_vel` drives the prediction, `/landmark_detections` the correction; publishes `/slam_pose` and `/slam_landmarks`.

Question C — D* Lite Incremental Replanning

A shelf-aisle occupancy grid (15 × 12 cells) in which each shelf wall has one convenient upper cross-aisle gap, with a long bottom bypass corridor as the only alternative. D* Lite searches *backwards from the goal* and caches its `g` / `rhs` values, so an edge-cost change repairs only the affected part of the search tree.

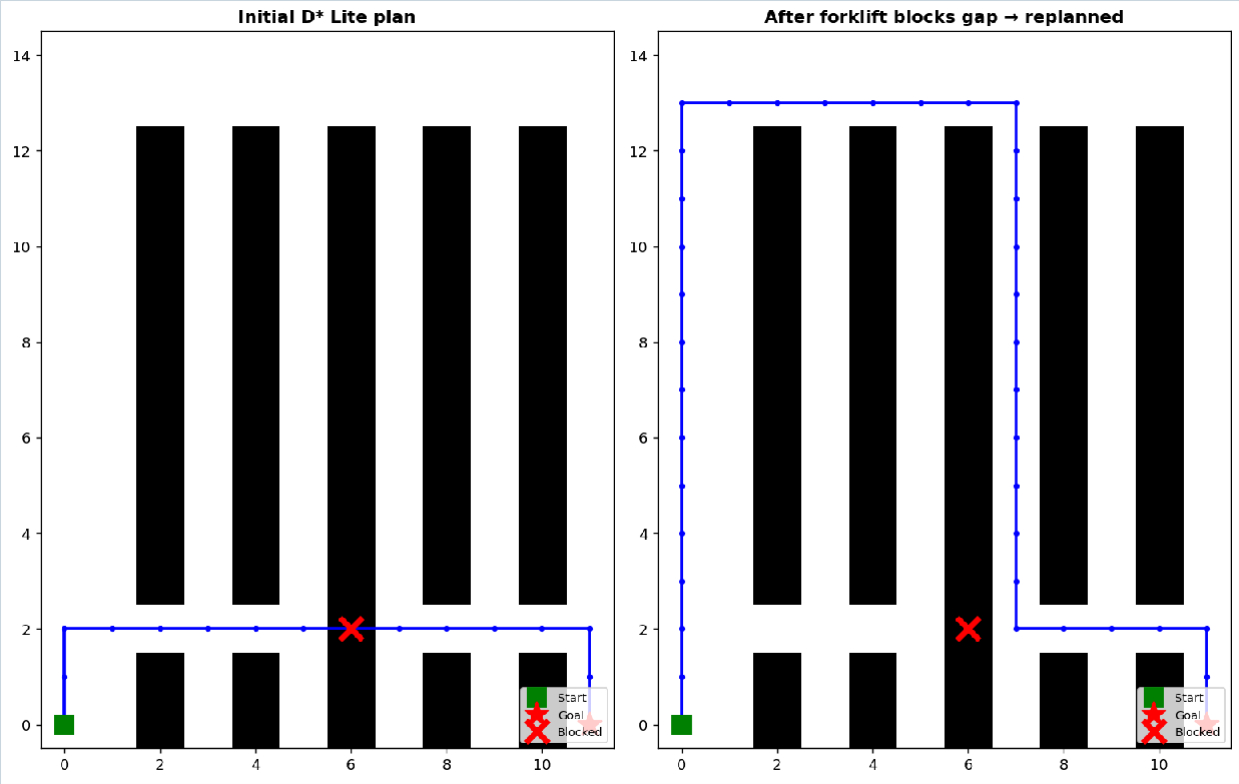


Figure 3 — `C_dstar.png`, *generated by* `Warehouse_bot_Assignment2_Group11.ipynb`.

DESCRIPTION & OBSERVATIONS

- **Left** — the initial route slipping through the nearest upper gap: **16 cells**.
- **Right** — after a forklift blocks the gap at cell **(2, 6)** (red cross), the planner reroutes down the bottom bypass: **38 cells**.
- **Result:** the reroute is produced by repairing a **single changed vertex and its neighbours**, not by re-solving the grid — the incremental property that makes D* the standard choice over re-running A* whenever the warehouse changes.
- **ROS 2 node:** `/dstar_planner` — subscribes `/map` (`nav_msgs/OccupancyGrid`, transient-local) and `/goal_pose`, publishes `/plan` (`nav_msgs/Path`).

Question D — Reinforcement Learning for Active Localization

Classical MCL is *passive*: it fuses whatever measurements arrive along a route chosen by someone else. Framing localization as a belief-MDP and solving it with tabular Q-learning yields *active* MCL, where the reward is the localizer's own health — the drop in belief entropy per step, minus a step cost, plus a bonus for crossing the localized threshold.

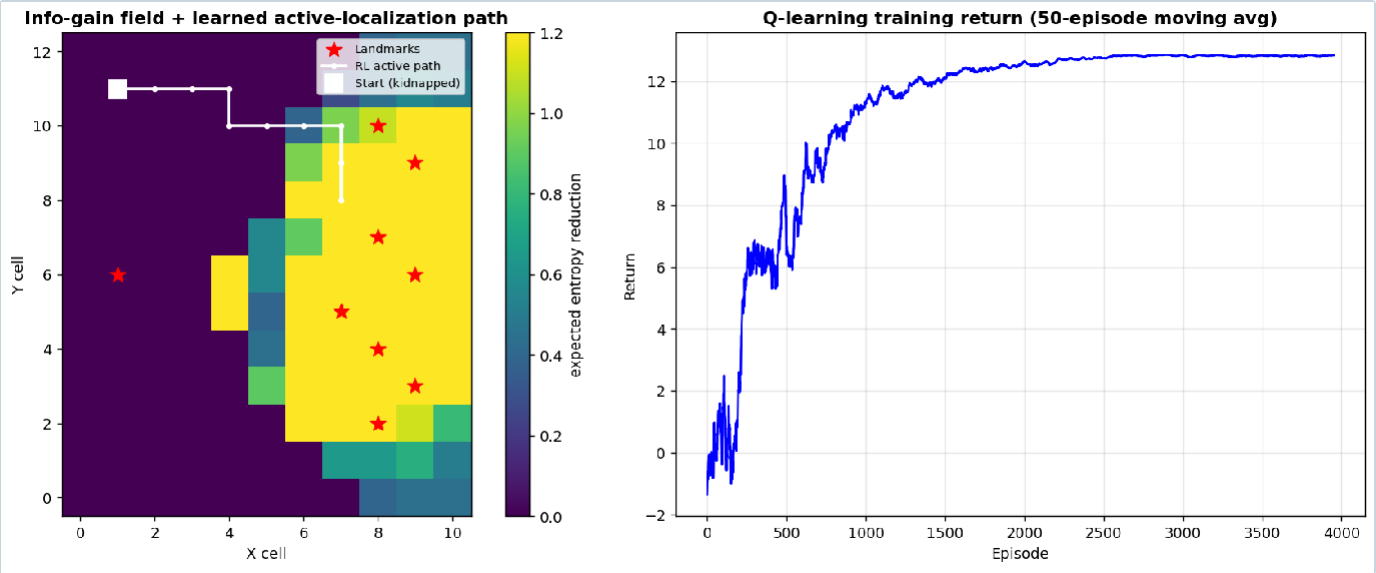


Figure 4 — `D_rl.png`*, generated by* `Warehouse_bot_Assignment2_Group11.ipynb`*.*

DESCRIPTION & OBSERVATIONS

- **Left** — the expected-entropy-reduction (information-gain) field, with the feature-rich QR cluster on the right and the ambiguous, self-similar aisle on the left; the white line is the learned policy's path from the 'kidnapped' start.
- **Right** — Q-learning training return over 4000 episodes (50-episode moving average).
- **Result:** the learned policy localizes in **9 steps** against **69.6 steps** for an undirected random walk — a **7.7×** speed-up, achieved by driving out of the aliased aisle into the QR-dense block.
- **ROS 2 node:** `/active_localization` — subscribes `/localization_entropy`, answers on `/cmd_vel`, closing the perception–action loop through the middleware.

Question E — Traditional ML Replacing the SLAM Pipeline

The entire EKF back-end — motion Jacobian, measurement Jacobian, inverse-observation model, data association and $O(n^2)$ covariance updates — is replaced by a supervised regression that learns the inverse sensor map directly, from a fixed-length observation descriptor [visible, range, sin b, cos b] per QR tag to the global pose. This is the fingerprinting view of localization, realistic for a warehouse whose shelving can be surveyed once offline.

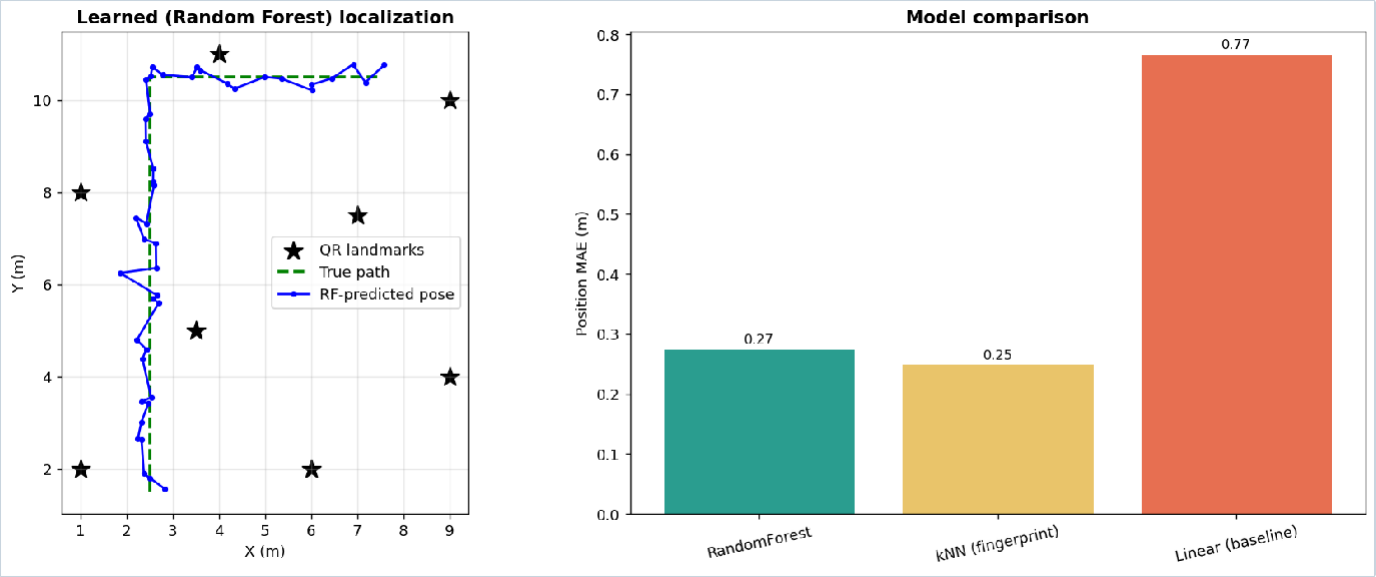


Figure 5 — `E_ml.png`*, generated by* `Warehouse_bot_Assignment2_Group11.ipynb`*.*

DESCRIPTION & OBSERVATIONS

- **Left** — the Random Forest's predicted poses tracking the true path with no filter and no Jacobians.
- **Right** — model comparison on 1500 held-out samples.
- **Result:** Random Forest **0.274 m** / 3.8° and k-NN fingerprint **0.249 m** / 5.4° position MAE — on par with the EKF-SLAM of Part B — while the linear baseline collapses to **0.766 m** / 19.5°, empirically confirming the non-linearity that forces the EKF to linearize. Deployed along a trajectory the forest achieves **0.230 m** mean error.
- **ROS 2 node:** `/learned_localizer` — honours the *same* topic contract as `/mcl_localizer` and `/ekf_slam`, so swapping estimators is a launch-file edit.

ROS 2 Deployment — Node and Topic Graph

An `rqt_graph`-style reconstruction of the computation graph actually exercised by the notebook, one panel per deliverable: publishers on the left, topics with their message counts in the middle, subscribers on the right. Every node is written against the real `rclpy` API and exchanges standard ROS 2 interfaces — `nav_msgs`, `sensor_msgs`, `geometry_msgs`, `std_msgs` and `tf2_ros` — over the REP-105 frame tree `map → odom → base_link`.

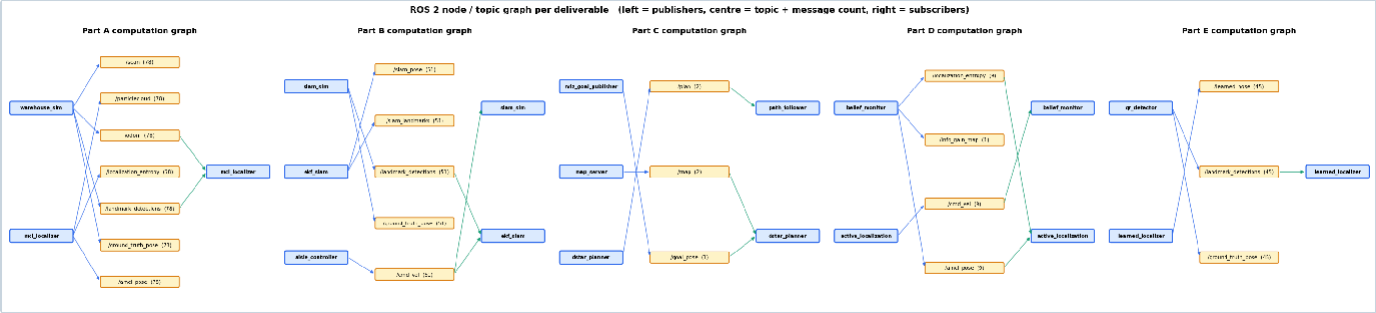


Figure 6 — `F_rosgraph.png`, generated by `Warehouse_bot_Assignment2_Group11.ipynb`.

DESCRIPTION & OBSERVATIONS

- **Result: 13 nodes** across the five deliverables exchanging **1,143 messages** over **16 distinct topics**.
- Two QoS profiles follow ROS 2 convention: best-effort for high-rate sensor streams, and reliable *transient-local* for the map, so a planner that starts late still receives the last grid.
- Because each deliverable is a node with an explicit interface contract, the parts compose rather than merely coexist — Part D's policy consumes the entropy Part A publishes, and Part E's learned estimator is interchangeable with Part B's EKF.

Screenshot and video of assignment from virtual labs

The screen recordings and screenshots captured while performing the assignment on the **Virtual Labs** platform are hosted on Google Drive at the link below. The recording documents the experiment being carried out end to end, together with the intermediate screens and the observations captured during the session.

GOOGLE DRIVE — SCREENSHOTS & VIDEO RECORDING

https://drive.google.com/file/d/1f3YOFpdtKWlq5vAKc0l-bXOQqsALVRsb/view?usp=sharing

ACCESS DETAILS

ITEM	DETAIL
Content	Screenshots and video recording of the assignment performed on Virtual Labs
Hosted on	Google Drive (shared link)
File ID	`1f3YOFpdtKWlq5vAKc0l-bXOQqsALVRsb`
Access	Anyone with the link can view
Submitted by	Group 11 — AI, ML & Robotics

> **Note.** The link above is clickable in this PDF. If your reader does not open it directly, copy the full URL into a browser address bar. Should access ever be restricted, please request permission through the Drive link and it will be granted to the evaluator.